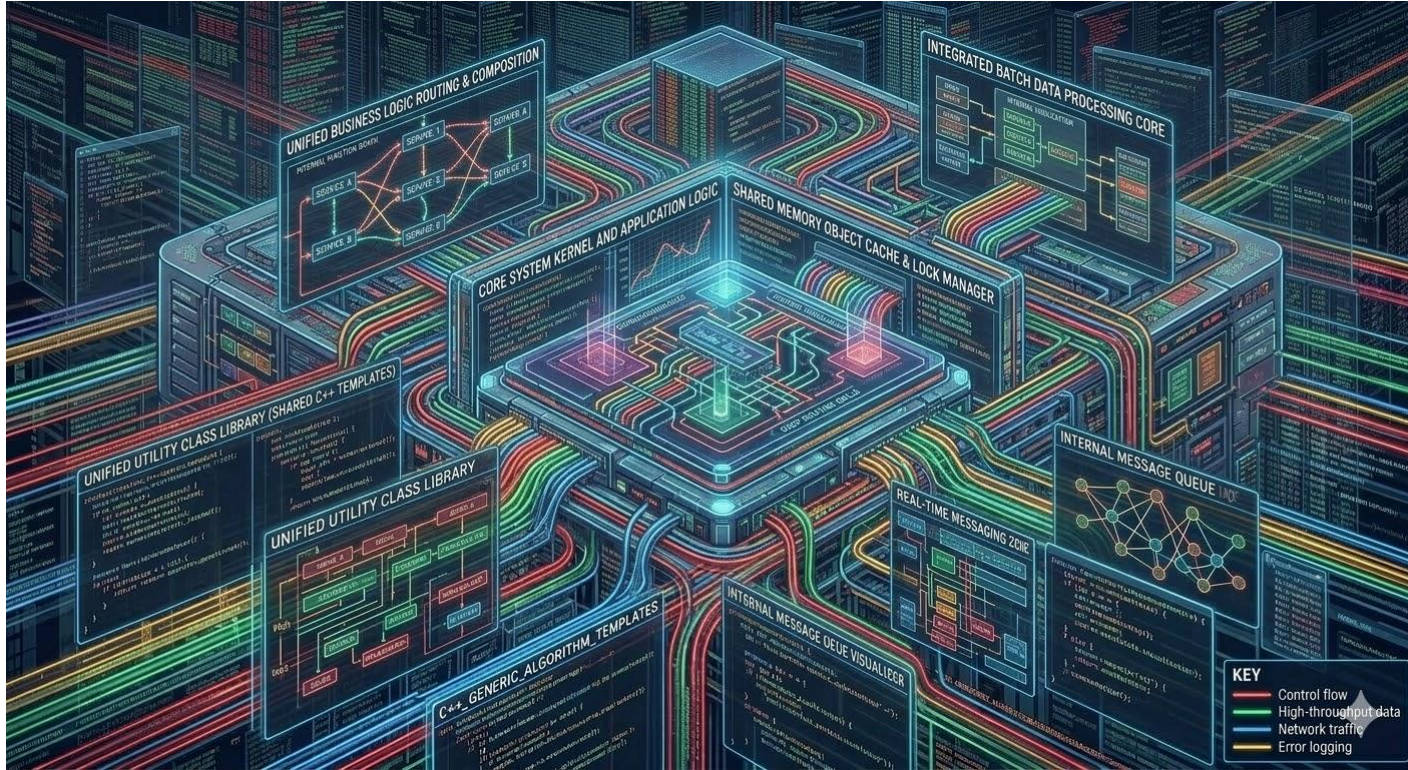


CS3213: Foundations of Software Engineering

Program Slicing



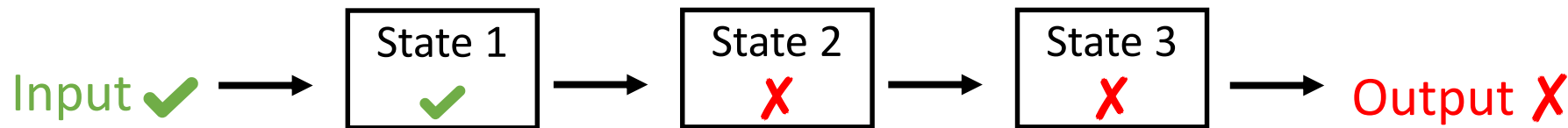
(Image by Gemini)



Program Slicing

- Question: how to automatically determine which earlier variables influenced the current (erroneous) state?
- **Slicing**: extracts a subset of the program that (potentially) affects variables or values at a certain program location
- Main advantage: we only need to inspect the slice when debugging, rather than the whole program
- Helps us both as a **mental model** and if implemented as a **tool**

Tracking Origins (1/2)



Mistake → Defect → Fault → ... → Fault → Failure

- Debugging strategy: “tracking origins”
- We start with a single faulty state f – the failure.
- We determine f 's **origins** – the parts of earlier states that could have caused the faulty state f .
- For each of these origins e , we determine whether they are faulty or not, until we have found the defect.

Tracking Origins

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

```
int m = middle(2, 1, 3);  
System.out.println(m);
```

incorrectly prints 1

Tracking Origins

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

```
int m = middle(2, 1, 3);  
System.out.println(m);
```

incorrectly prints 1

Where *could* the incorrect value stem from?

Ruling Out Program States

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

```
int m = middle(2, 1, 3);  
System.out.println(m);
```

A *slice* identifies the subset of the program that could have influenced the variable/returned value



Advantage of Slices

- *Rule out* those locations of the program that could *not* have an effect on the failure.
- Bring together possible origins that may be scattered across the code.

Ruling Out Program States

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

```
int m = middle(2, 1, 3);  
System.out.println(m);
```

A *slice* identifies the subset of the program that could have influenced the variable/returned value

How can we automatically compute a slice?

Tracking Origins

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

The conditions $y < z$, $x \geq y$ and $x < z$ are origins of the returned value and, in turn, have x , y , and z as origins

Tracking Origins

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

Control conditions correspond
to *control dependencies*

Tracking Origins

```
public static int middle(int x, int y, int z) {  
    if (y < z) {  
        if (x < y) {  
            return y;  
        } else if (x < z) {  
            return y;  
        }  
    } else {  
        if (x > y) {  
            return y;  
        } else if (x > z) {  
            return x;  
        }  
    }  
    return z;  
}
```

Earlier data values correspond
to *data dependencies*

Two Types of Dependencies

- We can reason about program dependencies using *a program dependence graph*
- We distinguish two kinds of dependencies $x \leftarrow y$
 - **Data dependency:** x is assigned a value computed from y . In our example, m is data dependent on the return value of `middle()`
 - **Control dependency:** A statement involving x is executed only because a condition involving y was evaluated, influencing the execution path. In our example, the value returned by `return y` is control dependent on the several conditions along its path, which involve x , y , and z .

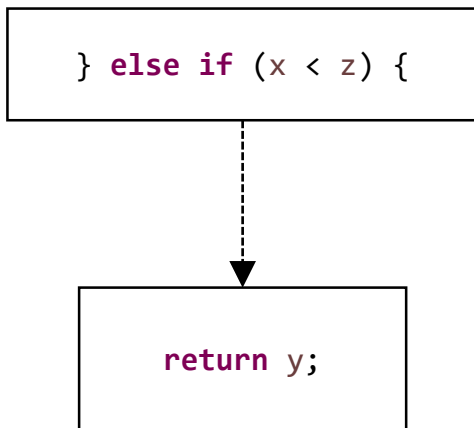
Data Dependence

```
public static int middle(int x, int y, int z) {
```

```
    return y;
```

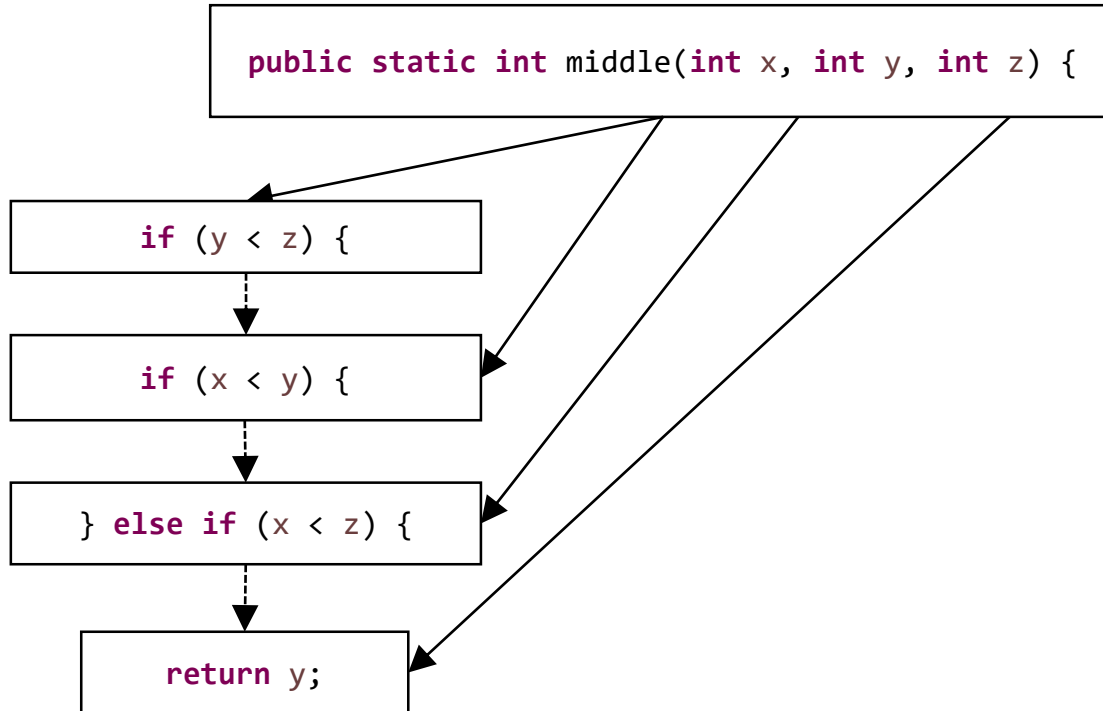
The **value y** in the return statement is **data dependent on the y** that is passed to the middle method (i.e., the value of y in the upper node flows to the y in the lower node)

Control Dependence



The execution of the return statement is controlled by the earlier test `x < z` (i.e., control dependent). We use gray dashed lines to indicate control dependencies.

Program Dependence Graph



Static vs. Dynamic Slicing (1/5)

- Program slicing was proposed in 1984 by Mark Weiser
- First program slicing approach was static
 - Given a variable at some program statement that is observed to be incorrect, what is the relevant subset of the program (i.e., slice)?

352

IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, VOL. SE-10, NO. 4, JULY 1984

Program Slicing

MARK WEISER

Abstract—Program slicing is a method for automatically decomposing programs by analyzing their data flow and control flow. Starting from a subset of a program's behavior, slicing reduces that program to a minimal form which still produces that behavior. The reduced program, called a "slice," is an independent program guaranteed to represent faithfully the original program within the domain of the specified subset of behavior.

Some properties of slices are presented. In particular, finding statement-minimal slices is in general unsolvable, but using data flow analysis is sufficient to find approximate slices. Potential applications include automatic slicing tools for debugging and parallel processing of slices.

Index Terms—Data flow analysis, debugging, human factors, parallel processing, program maintenance, program metrics, slicing, software tools.

INTRODUCTION

LARGE computer programs must be decomposed for understanding and manipulation by people. Not just any decomposition is useful to people, but some—such as decomposition into procedures and abstract data types—are very useful. Program slicing is a decomposition based on data flow and control flow analysis.

A useful program decomposition must provide pieces with predictable properties. For instance, block-structured languages [17] are powerful in part because their scope and control flow rules permit understanding procedures independent of context. Similarly, abstract data type languages [12], [15], [25] make further control and scope restrictions for even greater context independence. Therefore, the pieces of a program decomposed by dataflow, i.e., the "slices," should be related to one another and the original program in well defined and predictable ways.

As we will see, slices have a very clear semantics based on projections of behavior from the program being decomposed. Unlike procedures and data abstractions, slices are designed to be found automatically after a program is coded. Their usefulness shows up in testing, parallel processor distribution, maintenance, and especially debugging. A previous study showed experienced programmers mentally slicing while debugging, based on an informal definition of slice [22]. Our concern here is with 1) a formal definition of slices and their abstract properties, 2) a practical algorithm for slicing, and 3) some experience slicing real programs.

DEFINITIONS

This section considers programs without procedure calls. Procedures are discussed later. The first few definitions review the standard definitions of digraph, flowgraph, and computation in terms of state trajectory. Finally, a slice is defined as preserving certain projections from state trajectories.

The next few definitions simply establish a terminology for graphs, and restrict attention to programs whose control structure is single-entry single-exit ("hammock graphs").

Definition: A digraph is a structure (N, E) , where N is a set of nodes and E is a set of edges in $N \times N$. If (n, m) is in E , then n is an immediate predecessor of m and m is an immediate successor of n . A path from n to m of length k is a list of nodes $p_0, p_1, \dots, p_k$ such that $p_0 = n$, $p_k = m$, and for all i , $1 \leq i \leq k-1$, (p_i, p_{i+1}) is in E .

Definition: A flowgraph is a structure (N, E, n_0) , where (N, E) is a digraph and n_0 is a member of N such that there is a path from n_0 to all other nodes in N . n_0 is sometimes called the initial node. If m and n are two nodes in N , m dominates n if m is on every path from n_0 to n .

Definition: A hammock graph is a structure (N, E, n_0, n_e) with the property that (N, E, n_0) and (N, E^{-1}, n_e) are both flowgraphs. Note that, as usual, $E^{-1} = \{(a, b) | (b, a) \text{ is in } E\}$. If m and n are two nodes in N , m inverse dominates n if m is on every path from n to n_e .

In the remainder of the paper, all flowgraphs will be assumed to be hammock graphs. In addition to its flowgraph, every program is assumed to provide the following information.

Definition: Let V be the set of variable names which appear in a program P . Then for each statement n in P (i.e., node in the flowgraph of P) we have the following two sets, each a subset of V : $REF(n)$ is the set of variables whose values are used at n , and $DEF(n)$ is the set of variables whose values are changed at n .

A state trajectory of a program is just a trace of its execution which snapshots all the variable values just before executing each statement.

Definition: A state trajectory of length k of a program P is a finite list of ordered pairs

$$(n_1, s_1)(n_2, s_2) \cdots (n_k, s_k)$$

Static vs. Dynamic Slicing (2/5)

- The first dynamic slicing approach was proposed in 1990
 - Given a variable at some program statement that is observed to be incorrect **as well as concrete execution**, what is the relevant subset of the program (i.e., slice)?
- Typically a better fit for debugging

Dynamic Program Slicing*

Hiralal Agrawal

Department of Computer Sciences
Purdue University
West Lafayette, IN 47907-2004

Joseph R. Horgan

Bell Communications Research
Morristown, NJ 07960-1910

Abstract

The conventional notion of a program slice—the set of all statements that *might* affect the value of a variable occurrence—is totally independent of the program input values. Program debugging, however, involves analyzing the program behavior under the specific inputs that revealed the bug. In this paper we address the dynamic counterpart of the static slicing problem—finding all statements that *really* affected the value of a variable occurrence for the given program inputs. Several approaches for computing *dynamic slices* are examined. The notion of a Dynamic Dependence Graph and its use in computing dynamic slices is discussed. We introduce the concept of a Reduced Dynamic Dependence Graph whose size does not depend on the length of execution history, which is unbounded in general, but whose size is bounded and is proportional to the number of dynamic slices arising during the program execution.

Uses of program slicing have been suggested in many applications, e.g., program verification, testing, maintenance, automatic parallelization of program execution, automatic integration of program versions, etc. (see, e.g., [Wei84, HPR89]). In this paper we are primarily concerned with its use in program debugging [Wei82]. Often during debugging the value of a variable, *var*, at some program statement, *S*, is observed to be incorrect. Program slicing with respect to *var* and *S* gives that relevant subset of the program where one should look for the possible cause of the error. But the above notion of program slicing does not make any use of the particular inputs that revealed the error. It is concerned with finding all statements that *could* influence the value of the variable occurrence for *any* inputs, not all statements that *did* affect its value for the *current* inputs. Unfortunately, the size of a slice so defined may approach that of the original program, and the usefulness of a slice in debugging tends to diminish as the size of the slice increases. Therefore, in this paper we examine a narrower notion of “slice,” consisting only of state-

1 Introduction

Static vs. Dynamic Slicing (3/5)

```
private static boolean isEven(int num) {  
    boolean even;  
    if (num % 2 == 1) {  
        even = true;  
    } else {  
        even = false;  
    }  
    return even;  
}
```

What statements might have caused the incorrect value of even?

Static vs. Dynamic Slicing (4/5)

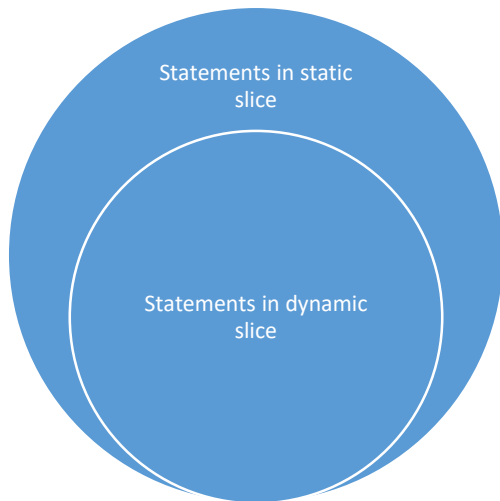
```
private static boolean isEven(int num) {  
    boolean even;  
    if (num % 2 == 1) {  
        even = true;  
    } else {  
        even = false;  
    }  
    return even;  
}
```

Static slicing: the slice
contains the whole method

Static vs. Dynamic Slicing (5/5)

```
private static boolean isEven(int num) {  
    boolean even;  
    if (num % 2 == 1) {  
        even = true;  
    } else {  
        even = false;  
    }  
    return even;  
}  
  
isEven(1)
```

Dynamic slicing: the else branch was not executed, and cannot be part of the slice





Forward vs. Backward Slicing

- Forward and backward slicing
 - Backward slicing: what influenced a value?
 - Forward slicing: what statements are influenced by the value?



Slicing Applications

- Debugging strategy!
- Some support in IDEs
- Application areas: debugging, program understanding and maintenance, security analysis, ...



Summary and Takeaways

- “Tracking origins” debugging strategy
- Slicing
- Static and dynamic slicing
- Forward and backward slicing